How much junk do rules catch?
pygarble on real survey responses and social-media text

Ujjwal Rao
[TODO: affiliation]
[TODO: email]
`https://github.com/brightertiger/pygarble`

October 2026

Abstract

Free-text fields collect junk: keyboard mashing, random strings and placeholder answers. pygarble is a Python library with no runtime dependencies that flags such text with rules and small embedded English statistics. We measure it on two public datasets against a Markov-chain gibberish detector and a language identifier, with every method at its own default decision. On 3,513 survey responses labelled as junk, pygarble's `english_fusion` profile flags 0.895. On 9,971 English rows of random strings and tweets, it recalls 0.991 of the random strings with precision 0.906, from a 1.05 MB install. A post hoc variant of the default profile without its two encoding-damage checks shows that the ensemble can be adapted to a domain by removing strategies: its false-positive rate on tweets falls from 33.2 % to 18.2 % with no loss of recall. We report where each method is beaten, its cost, and which configuration to use for which job.

1 Introduction

Text pipelines receive junk. Survey free-text questions get keyboard mashing from respondents who want to reach the next page. Sign-up forms get random strings from bots. Scraped text carries encoding damage. Each of these costs something downstream: a coder reads it, a model is trained on it, or an analysis counts it as an answer.

The usual remedies are heavy. A language model or a trained classifier needs a model download, a GPU or an API call, and its decisions are hard to explain row by row. Simple rules are cheap, local and explainable, and are often the first filter a team writes by hand. The question is how much junk such rules actually catch, and how much good text they wrongly flag.

pygarble [3] packages rules of this kind: character and word statistics computed against embedded English tables, with no runtime dependencies. This paper measures it on real data rather than on examples written by its author. **[TODO: one or two sentences on the maintainer's own motivating use case.]**

The contributions are:

- A benchmark on two public datasets: 3,513 survey responses labelled as junk, and 9,971 English rows of random strings and tweets. Every method runs at its own default decision. The protocol, a pilot run and every change after it are disclosed, and per-row predictions are saved so every number can be recomputed.
- A comparison of pygarble's default profile and its `english_fusion` profile with two CPU baselines: a character Markov-chain gibberish detector and a language identifier.

- A post hoc, domain-specific tuning of the default profile, reported separately from the pre-registered results.
- The cost of each method, a plain account of where pygarble is beaten, and guidance on which configuration to use.

2 pygarble in one page

pygarble targets Python 3.8 and later and has no runtime dependencies. The installed package, with its embedded English tables, is 1.05 MB. Besides gibberish detection it scans text for secrets, personal data and profanity, with redaction; those features are not evaluated here.

Strategies. Gibberish detection is built from 32 strategies, each a function from text to a score in $[0, 1]$ and a decision. They fall into a few groups: character statistics (vowel ratio, entropy, consonant runs, letter frequency); English character models (a Markov chain, bigram and trigram tables, a log-likelihood ratio against a uniform reference); word-level checks (dictionary lookup, word anomaly, function-word density); structural checks (repetition, keyboard rows and adjacency, hex strings); encoding checks (mojibake, control characters, Unicode script mixing); and four checks that compare a text window with frequent English (cross parsing, primed compression, n-gram rank and a permutation test). Each strategy reports when it does not apply, for example to text with no Latin letters.

Ensembles and profiles. `EnsembleDetector` combines a named set of strategies, a *profile*, under a voting rule, and flags a text when the combined score reaches the threshold, 0.5 by default. Two profiles are measured here:

- `english`, the default: Markov chain, log-likelihood ratio, word anomaly, mojibake, keyboard adjacency and control characters, with `any` voting, so one member firing flags the text.
- `english_fusion`, opt-in: word lookup, log-likelihood ratio and cross parsing with `fisher` voting. Each member's score is read against that member's scores on a synthetic English null built from word frequencies, and the tail probabilities are combined with Fisher's method.

Other profiles target encoding damage, Unicode spoofing and degenerate model output; they are not measured here. A user can also pass any list of strategies, which is how the tuned variant in Section 5.1 is built.

What it is not. The English profiles score English plausibility. They are not language identifiers: meaningful text in other languages written in Latin letters can be flagged, and text in other scripts gives them no signal. Scores are heuristic, not calibrated probabilities. A `calibrate()` helper picks a threshold from a user's own labelled examples; this paper does not use it.

3 Datasets and protocol

A: survey junk. The Kaggle dataset `johnwdata/gibberish-text-classification` [1] has 3,767 survey responses, all labelled as junk. Despite its `.xls` extension, the file is a Windows-1252 CSV. We strip each response, drop empty ones and drop exact duplicates, leaving 3,513 rows. A has no clean rows, so the only metric is the *capture rate*: the share of rows a method flags. Its licence is listed as unknown, so the repository stores row indices and hashes, never the text.

Table 1: Capture rate on A and precision, recall and FPR on B at each method's default decision, with 95 % bootstrap intervals. The last row was constructed after the results were seen and is not a pre-registered result.

Method	A capture	B precision	B recall	B FPR
pygarble english (default)	0.830 [0.817, 0.843]	0.656 [0.645, 0.670]	0.947 [0.940, 0.954]	0.332 [0.320, 0.344]
pygarble english_fusion	0.895 [0.884, 0.905]	0.906 [0.898, 0.915]	0.991 [0.987, 0.993]	0.068 [0.062, 0.075]
gibberish-detector	0.722 [0.707, 0.736]	0.995 [0.993, 0.997]	0.822 [0.809, 0.833]	0.003 [0.002, 0.004]
lingua (not English)	0.989 [0.985, 0.992]	0.876 [0.867, 0.886]	0.994 [0.991, 0.996]	0.094 [0.087, 0.101]
Post hoc, domain-specific tuning				
pygarble english, tuned (no mojibake, no control characters)	0.830 [0.817, 0.843]	0.777 [0.766, 0.789]	0.947 [0.940, 0.954]	0.182 [0.173, 0.192]

B: random strings and tweets. The Hugging Face dataset `Sowmya15/gibberish_multilingual` [4] (Apache-2.0) pools train, validation and test files. We keep the 9,971 non-empty rows whose language is English: 4,000 random strings (junk) and 5,971 tweets (clean). Labels are used as given and duplicates are kept (215 rows repeat an earlier text). We report precision, recall and the false-positive rate (FPR), the share of tweets flagged.

Methods. pygarble 0.11.0 is used with its defaults frozen at commit `e3135ed`; no default or threshold was changed after any result was seen. The baselines run on CPU with pinned versions:

- **gibberish-detector** 0.1.1 [2], a character Markov model. The package ships no model, so we train one with its own trainer on the corpus its README uses, and flag scores above its command-line default of 4.0.
- **lingua** 2.2.0 [5], a language identifier, under the policy "not detected as English is junk".

There are no neural baselines.

Decision and intervals. Every method is scored at its own default decision: the binary output it gives with default settings. No threshold is fitted on either dataset. Intervals are 95 % percentile intervals from 1,000 bootstrap resamples of rows, drawn with a fixed seed and shared by all methods.

Pilot and changes. Before the protocol was written, all of A and all English rows of B were scored once with pygarble's defaults. The `english_fusion` numbers from that pilot are the ones reported here, because nothing in the package changed afterwards. The first version of the protocol also fitted thresholds at fixed false-positive rates on a development split of B and compared more baselines and pygarble configurations; both were removed from scope before this paper was written (Section 9). The tuned variant in Section 5.1 was added after the results were seen.

4 Results on survey junk

`english_fusion` flags 0.895 of the real survey junk in A (95 % interval 0.884–0.905), with no dependencies and a 1.05 MB install (Figure 1, left; Table 1). That is more than the Markov-chain gibberish-detector, which flags 0.722. The default `english` profile flags 0.830.

lingua flags 0.989, the most of any method. It does so by calling anything it cannot identify as English junk, a policy whose cost appears on B and in Section 7.

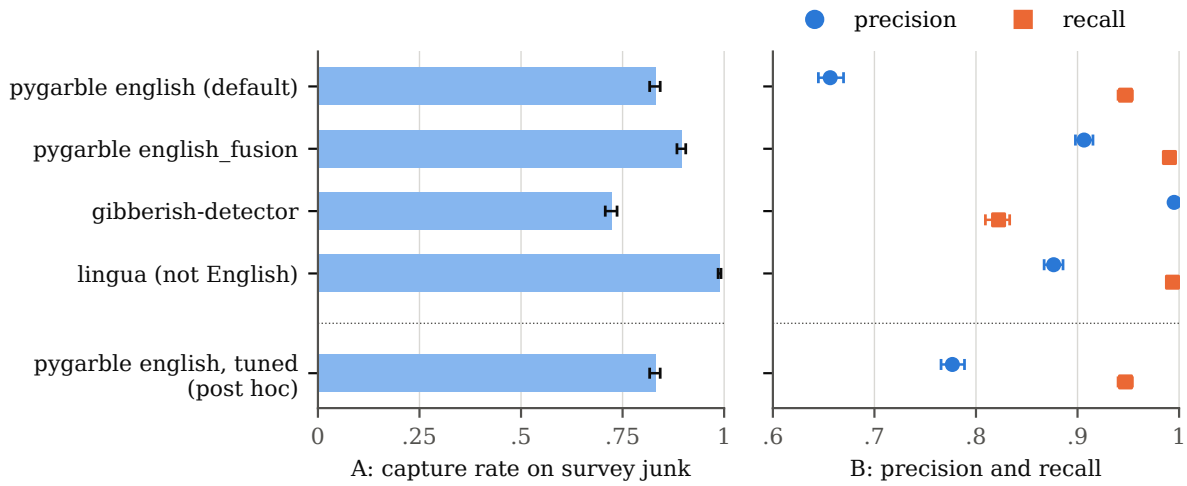


Figure 1: Left: capture rate on A (3,513 survey responses labelled as junk). Right: precision and recall on B (4,000 random strings, 5,971 tweets). Every method at its default decision, with 95 % bootstrap intervals. The tuned row below the dotted line is post hoc (Section 5.1).

5 Results on social-media text

On B, `english_fusion` recalls 0.991 of the random strings with precision 0.906 (Figure 1, right). It flags 6.8 % of tweets. lingua has similar recall (0.994) and lower precision (0.876). gibberish-detector has the highest precision, 0.995, and the lowest recall, 0.822. The default `english` profile recalls 0.947 with precision 0.656.

A note on what counts as a false positive. 26.4 % of the tweets in B contain mojibake byte signatures, for example a UTF-8 emoji decoded as Windows-1252. B labels these tweets clean, because it was built to separate random strings from tweets. A pipeline that audits encoding would want them flagged.

5.1 Domain-specific tuning

The default `english` profile includes two strategies that target encoding damage: mojibake and control characters. The second fires on the C1 control characters inside mojibake. Since this dataset labels encoding damage as clean, a user screening social-media text for gibberish would drop both. We built that variant: the `english` profile without mojibake and control characters, with the same `any` voting and threshold. The package was not changed.

This variant was constructed after seeing that 26.4 % of the tweets carry mojibake and that these strategies flag them. It is post hoc and reported as such, not as a pre-registered result. Its results next to the untuned profile:

	A capture	B precision	B recall	B FPR
`english`	0.830	0.656	0.947	0.332
`english`, tuned	0.830	0.777	0.947	0.182

Capture on A and recall on B are unchanged, because no junk row in either dataset depended on the two encoding checks. Precision rises and the FPR on tweets falls. The point is not this particular variant but that the ensemble can be adapted to a domain by removing strategies whose

Table 2: Footprint. Time is the default decision over all 9,971 B texts on one thread of an Apple M2 with Python 3.12.2, median of five runs after a warm-up. Install size counts the installed files of the distribution and its runtime dependencies, plus the training corpus gibberish-detector's model is built from.

Method	ms per 1,000 texts	Install (MB)
pygarble english (default)	66.4	1.05
pygarble english_fusion	62.0	1.05
gibberish-detector	6.8	6.51
lingua (not English)	1039.0	307.47
pygarble english, tuned (no mojibake, no control characters)	52.7	1.05

target the domain does not treat as junk. `english_fusion` has no mojibake member, so it has no such variant.

6 Cost

Table 2 lists time per 1,000 texts and install size. pygarble scores 1,000 texts in 62.0 ms with `english_fusion` and installs in 1.05 MB with nothing else. gibberish-detector is faster, 6.8 ms, at 6.51 MB. lingua takes 1039.0 ms and 307.47 MB. All methods run offline once installed. Timings depend on the machine; the ordering is what carries over.

7 Where it fails

False positives. The Markov baseline, gibberish-detector, flags 0.3 % of tweets; `english_fusion` flags 6.8 %.

The default profile on informal text. The default `english` profile flags 33.2 % of tweets. Tuning lowers this to 18.2 %, still above `english_fusion`.

Language identification captures more survey junk. lingua flags 0.989 of A against fusion's 0.895, at an FPR of 9.4 % on tweets and a 307.47 MB install.

Short inputs and word-like mashes. `english_fusion` misses 369 of the 3,513 rows in A. On inspection, many are very short: single letters, digits, punctuation or a few repeated letters. The rest are longer strings built from real words or pronounceable fragments, which read as English to character and word statistics.

Label noise in A. A is used as labelled. 15 rows are flagged by none of the four untuned methods. On inspection many are meaningful but low-effort answers, such as thanks or a short positive phrase, so A's capture rates have a ceiling below one.

B is easy. B's junk is random character strings, which share almost nothing with English; fusion and lingua recall 0.991 and 0.994 of them. B therefore says little about recall on realistic junk and mostly measures false positives on informal English.

8 Guidance

- **Screening survey or form free text:** `english_fusion`. It catches 0.895 of real survey junk and 0.991 of random strings, at 62.0 ms per 1,000 texts, from a 1.05 MB install with no dependencies. Add a minimum-length rule if one- and two-character answers count as junk.
- **Adapting to a domain:** start from a profile and remove the strategies whose target the domain does not treat as junk, as in Section 5.1. For social-media text, drop mojibake and control characters unless encoding damage is itself a concern.
- **When false positives cost much more than misses:** at default settings, gibberish-detector flags the fewest tweets. Alternatively, choose an `english_fusion` threshold with `calibrate()` on a few hundred labelled rows of your own data; this paper does not measure that option.
- **Multilingual input:** route by language first. A language identifier answers that question; it is not a junk detector.

9 Limitations

Two public datasets are measured, both English. A has only junk, so it measures capture and nothing about false positives; B's junk is random strings, which are easier than real junk. Both datasets are short texts; pygarble's behaviour on long documents is not measured here. There is no neural baseline, and only two baselines. A comparison at a matched false-positive rate was computed in PR 18 and removed from scope at the maintainer's direction; it remains available in the repository history. Each method is run once at its default decision, so the results say how the methods behave out of the box, not how well they can be tuned. The one tuned variant is post hoc and was chosen with knowledge of B.

10 Reproducibility

The benchmark code is in `paper/scripts/benchmark/`, the protocol and every deviation from it in `paper/benchmark/`, and results in `paper/benchmark/results/`. The manifest records the sha256 hash of each raw file (for A, beginning `091d5d3895f035d5`) and every row id; no dataset text is committed. Per-row scores and decisions for every method are committed, so every number in this paper can be recomputed without the raw data:

```
make benchmark-verify PYTHON=<venv>/bin/python
make preprint PYTHON=<venv>/bin/python
```

The first command recomputes `metrics.json` from the saved predictions and fails on any difference. The second regenerates the figure, the tables and the number macros this paper is typeset from, then builds the PDF. Re-scoring from the raw files is described in `paper/scripts/benchmark/README.md`; scores are deterministic.

AI assistance

Claude Code agents running Claude Opus 5.5, Claude Sonnet 5.5 and Claude Fable 5.1 wrote the benchmark code, ran the experiments and drafted this text, under the author's direction and review. Commit trailers in the repository record the models used.

References

[1] johnwdata. Gibberish text classification. Kaggle dataset, https://www.kaggle.com/datasets/johnwdata/gibberish-text-classification, 2026. File Gibberish.xls; licence listed as Unknown; accessed 2026-10-11.

[2] Aaron Loo. gibberish-detector. https://github.com/domanchi/gibberish-detector, 2026. Version 0.1.1 on PyPI; a port of https://github.com/rrenaud/Gibberish-Detector; accessed 2026-10-11.

[3] Ujjwal Rao. pygarble: local screening of text for secrets, PII, profanity and gibberish. https://github.com/brightertiger/pygarble, 2026. Version 0.11.0, commit e3135ed.

[4] Sowmya15. gibberish_multilingual. Hugging Face dataset, https://huggingface.co/datasets/Sowmya15/gibberish_multilingual, 2026. Apache-2.0; accessed 2026-10-11.

[5] Peter M. Stahl. Lingua: the most accurate natural language detection library for Python. https://github.com/pemistahl/lingua-py, 2026. Package lingua-language-detector, version 2.2.0; accessed 2026-10-11.